



BUSINESS CENTRAL · AGENTIC DEVELOPMENT FRAMEWORK

Build AI the right way.

Simple enough for Functional Consultants. Robust enough for Pro Developers.

AJ ANSARI · MICROSOFT MVP

ajansari.github.io/ocpfBcAgenticDevFramework



AGENDA

01 **Background and origin**

02 **The pipeline, step by step**

03 **Runbook approach and model roles**

04 **Continuous discipline and documents**

05 **Full and Lite editions**

06 **Getting started with the plugins**

A framework for building AL extensions with AI, the way a professional developer would

BUILT FOR

Functional Consultants and other non-developers. Professional AL developers will find it just as useful.

GROUNDING IN

The AL MCP Server, BC Base and System App docs on Microsoft Learn, BCQuality, and the OCPF AL Development Standards Guide.

TWO EDITIONS

Full (14 steps) for substantial projects. Lite (7 steps) for smaller ones that need to move fast. Same AL standards in both.

Interface with an agent the way you would with a BC developer

The spark was Microsoft's **Business Central Agentic Engineering Process** vision, unveiled at Directions North America in April 2026.

Vibe coding brings ill effects to business-critical code. The goal was a framework that lets Functional Consultants use agentic tools to build AL not just quickly, but the **right** way.

Expect the same quality of work and the same outputs a human BC developer would deliver, without unnecessary technical debt.

From bootcamp to plugin

Since 2023

AL development bootcamp at Community Summit NA, plus AL workshops for Functional Consultants.

April 2026

Microsoft unveils the BC Agentic Engineering Process vision at Directions NA, Orlando.

May – June 2026

First proof of concept around the AL Standards Guide; first proper version by early June.

August 2026

External debut at Days of Knowledge ANZ in Melbourne, to very positive feedback.

Since

Lite edition, AL MCP Server and BCQuality integration, VS Code and CLI plugins for Claude Code and GitHub Copilot.

Grounded in official references, not guesswork

OCPF AL Development Standards Guide

The detailed AL rules the routine applies at each step. Fetched at PRE-01.

BC Base & System App on Microsoft Learn

Official references instead of half-remembered platform behavior.

BCQuality

A curated quality knowledge base, snapshotted per project.

OCPF BC AL Patterns Library

A portable pattern library, consulted before diagnosing from scratch.

AL MCP Server

Build, publish, symbol and diagnostic tools, driven directly.

AL Guidelines

Community-driven, Microsoft-hosted AL best practices.

Translation & Multi-language Support

Captions and tooltips as translatable text with XLIFF files, never hard-coded strings.

02

The pipeline

Four phases. Fourteen steps. A gate
on every one.

Nothing advances until the gate above it is met

PHASE 1 · PRE-01 – 01

DEFINE

Scope the problem, fill in parameters. Ask, don't infer.

PHASE 2 · 02 – 04

DESIGN

What, then how: an FRD in business language, then a TDD that stands on its own.

PHASE 3 · 05 – 07

BUILD

Generate AL, lint, then compile and package. Symbol-verified lint, no per-batch compile.

PHASE 4 · 08 – 12

PROVE

Gap-fit, review, document, release, until the package that was tested is the one in production.

In Production, Documented.

Scope the problem, fill in parameters

PRE-01

State the Problem

Fetch the AL Standards Guide, capture raw requirements verbatim.

→ ProblemStatement.md · ProjectProgress.md

GATE Functional Consultant signs off

PRE-02

Structured Gap Analysis

Analytical, posted, lookup, legacy vs. modern, tax, global/local.

→ expanded entity list + gap log

GATE Technical Lead reviews; gaps closed or deferred

01

Populate the Intake Sheet

Identity, ID ranges, naming/API, platform, onboarding, model & effort.

→ docs/ProjectParameters.md

GATE No placeholders; human confirms sheet

What, then how, before any code

02

Craft the FRD

Business language: what and why, not how.
Validated against DEFINE.

→ FRD.md

GATE Dev Manager sign-off; every entity accounted for

03

Craft the TDD

System identity, module grouping, batch plan, per-object spec, permission sets. Must be self-sufficient.

→ TDD.md

GATE Technical Lead sign-off; self-sufficiency check

04

Sanity Check

Can BC actually do this? Does the TDD implement the FRD?

→ SanityCheck.md

GATE 0 blocking issues

Plan, then generate batch by batch

05

Plan the Code

Batch order, scaffold, AL MCP Server + BCQuality + Patterns Library, pre-flight checks written.

GATE Batch order agreed; scaffold complete, not compiled

06 · repeats per batch

Code Generation

Main approves and generates, then runs pre-generation pre-flight. **Light** runs post-generation pre-flight and permission-set coverage, and reports back. Loop until every planned batch exists. No compile here.

GATE Every batch pre-flight clean; coverage verified

One mandatory compile opens the step. Then the loop runs until clean.

Reasoning diagnoses root cause: patterns library first, then the three questions. **Main** fixes.

Once clean, an optional agent-run API test pass against the sandbox, only if a live MCP connection exists; otherwise Postman or a low-code caller.

1 publish	2 test	3 diagnose
4 fix	5 repackaging	6 redeploy

↺ repeat until clean

GATE

0/0

errors / warnings

Prove the built code matches intent

08

Gap-Fit Test, Fidelity Validation

Three-way comparison: FRD vs. TDD vs. as-built.

GATE Every gap classified; fixes recompiled and repackaged

09

Code Review

Standards Anti-Patterns, BCQuality pass, dead code, permission-set re-verify.

GATE Critical findings resolved; fixes repackaged

Document, hand off, release

10

Update Design Documents

As-built PostDevTDD.md; FRD.md updated as the new baseline.

GATE As-built TDD regenerable; FRD reflects reality

11

Document the Code

Documentation · UserGuide · HumanUnitTestScript · Deployment, plus AutomatedTestScripts if asked for: AL Test Framework, API-level automation, or both.

GATE Docs generated from code; formal hand-off to the human

12

Release to Users for Testing

Real users run the scripts end to end on a sandbox.

→ docs/ReleaseTestResults.md

GATE All pass; that exact package deploys to Production

Fourteen gates, one rule: nothing advances until the gate is met

PRE-01	Functional Consultant signs off
PRE-02	Technical Lead reviews; gaps closed or deferred
01	No placeholders; human confirms sheet
02	Dev Manager sign-off; every entity accounted for
03	Technical Lead sign-off; self-sufficiency check
04	0 blocking issues
05	Batch order agreed; scaffold complete, not compiled

06	Every batch pre-flight clean; coverage verified
07	0 errors / 0 warnings
08	Every gap classified; fixes recompiled and repackaged
09	Critical findings resolved; fixes repackaged
10	As-built TDD regenerable; FRD reflects reality
11	Docs generated from code; formal hand-off to the human
12	All pass; that exact package deploys to Production

03

How it runs

One runbook powered by a backbone agent. Sub-agents spawned where needed.

Two ways to run an agentic pipeline

THIS FRAMEWORK

Runbook

One main agent is the backbone. It reads a single runbook, walks every step in order, and spawns ad-hoc sub-agents for specialized tasks or a second set of eyes, each with a different model and effort where that helps. One thread owns the code and the continuity documents from PRE-01 to production.

The runbook carries the discipline a professional developer has internalised: the sequence, the gates, the documents, and when to stop and ask. A Functional Consultant brings the business knowledge; the runbook supplies the workflow.

THE ALTERNATIVE

Orchestration loop

Multiple standalone agents, each with its own instructions and skills, invoked on demand or wired into an orchestration loop that hands work from one to the next.

A good fit for teams that already own the workflow and want to compose it themselves. The discipline lives in the orchestration and each agent's setup rather than in one file anyone can read, so it asks more of the person configuring it.

Set at Step 01, on the Intake Sheet

NOT CONFIGURED · THE DEFAULT

Everything runs through one model at the harness's own default effort.

CONFIGURED

A fixed three-role division, with model and effort asked separately, per role.

MAIN

LIGHT

REASONING

Only Main edits. Light and Reasoning report back.

MAIN

Primary Agent · a model like Claude Sonnet · effort High (default)

All code generation and every code edit, including applying what Light and Reasoning report. Owns ChangeLog, Object Register, ProjectMemory, TestingFeedback.md.

LIGHT

Sub-Agent · a model like Claude Haiku · effort High (default)

The post-generation pre-flight pass only: props, Rec.-qualification, dead code, indentation, coverage, symbol verification. Reports; never edits.

REASONING

Sub-Agent · a model like Claude Opus · effort High (default)

Sanity Check, Gap-Fit Test, Code Review, FRD and TDD authorship, root-cause troubleshooting. Drafts and diagnoses; never edits code or continuity docs.

Who acts at each step

PRE-01	State the Problem	Main
PRE-02	Gap Analysis	Main
01	Intake Sheet	Main
02	FRD	Reasoning drafts → Main integrates
03	TDD	Reasoning drafts → Main integrates
04	Sanity Check	Reasoning reviews → Main resolves
05	Plan the Code	Main

06	Code Generation	Main generates → Light pre-flights
07	Compile, Troubleshoot	Reasoning diagnoses → Main fixes
08	Gap-Fit Test	Reasoning compares → Main applies
09	Code Review	Reasoning reviews → Main applies
10	Update Design Docs	Main
11	Document the Code	Main
12	Release for Testing	Main coordinates; humans test

04

All along

Not a phase. Runs underneath all four,
every step, from PRE-01 to 12.

Twelve disciplines that never stop

01 Document Nineteen required documents, kept current, never retroactively.	02 Track Changes Every FRD/TDD deviation logged before the next batch.	03 Retain Explanations Who decided, and why, not just the outcome.	04 Testing Feedback Log Verbatim findings from Step 07 onward, triaged explicitly.
05 Project Memory docs/ProjectMemory.md: an anchor, not a narrative.	06 Project Progress ProjectProgress.md in the root, one row per step.	07 Packaging & Versioning outputAppPackage/, never deleted; Schema Sync Mode flagged per build.	08 Repository Hygiene What never reaches the client's remote, and what always does.
09 AL MCP Server Bootstrap once; prefer its tools over ad hoc terminal use.	10 BCQuality Snapshot Fetches once, outside the project root; refreshed only on request.	11 AL Patterns Library Consulted before diagnosing from scratch at Step 07.	12 AL Standards Guide Fetches at PRE-01. Unreachable means stop and ask, never recall it.

Nineteen documents, kept current, never retroactively

DEFINE

PRE-01 ProblemStatement.md

PRE-01 ProjectProgress.md

PRE-02 Gap log + entity list

01 docs/ProjectParameters.md

DESIGN

02 FRD.md

03 TDD.md

04 SanityCheck.md

ALL ALONG · OWNED BY MAIN

05+ ChangeLog

05+ Object Register

All docs/ProjectMemory.md

07+ TestingFeedback.md

All Roadmap

PROVE

10 PostDevTDD.md (as-built)

11 Documentation

11 UserGuide

11 HumanUnitTestScript

11 Deployment

11 AutomatedTestScripts (if asked for)

12 docs/ReleaseTestResults.md

Four rules the agent never bends

Fetch the Standards Guide. Never recall it.

PRE-01 fetches the guide from the repository into standardsGuide/. If it is unreachable, the agent stops and asks.

No hard-coded text.

Captions and tooltips as translatable text with XLIFF files, so the extension is ready for multiple languages from the first build.

Keep the client's remote clean.

standardsGuide/ is gitignored. The BCQuality snapshot lives outside the project root. outputAppPackage/ is never deleted.

Memory is an anchor, not a narrative.

docs/ProjectMemory.md and ProjectProgress.md let a fresh session pick up exactly where the last one stopped.

05

Editions

Not every extension needs
fourteen steps.

Lite reduces process, never the AL rules

FULL 14 steps · 19 documents

Past ~10 objects, multiple sign-off roles, the Main / Light / Reasoning split, or heading to AppSource, which tends to demand the fuller documentation trail.

LITE 7 steps · 4 documents

Roughly 5–10 AL files. One person driving, one AI model doing the work. No separate Dev Manager / Technical Lead / Functional Consultant sign-off roles.

Both editions fetch and apply the same OCPF AL Development Standards Guide. The same rules apply to a 5-file extension as to a 50-file one. One edition per project, never both at once.

What Lite merges, folds, and drops

FULL		LITE
FRD + TDD + Sanity Check	→	DesignDoc.md
Testing Feedback Log + Roadmap	→	ChangeLog.md
Multi-model role split	→	dropped

Nothing is lost by switching later. DesignDoc.md maps directly onto the full framework's TDD, and ChangeLog.md carries straight over. If a Lite project outgrows Lite mid-flight, swap in the full runbook.

06

Getting started

Install a plugin. Open your AL project.

Type one command.

Claude Code, in VS Code or the CLI

- 1 Install Claude Code: the VS Code extension, or the CLI in your terminal.
 - 2 Add the OCPF BC plugin, following the install instructions in the repository README.
 - 3 Open your AL project folder.
 - 4 Type **/ocpf-bc:start**, or just ask to start a BC project with the OCPF framework.
-

/ocpf-bc:start

Or: “*Start a Business Central project with the OCPF framework.*”

github.com/ajansari/ocpfBcAgenticDevFramework

GitHub Copilot, in VS Code or the CLI

- 1 Install GitHub Copilot: the VS Code extension with Copilot Chat, or the Copilot CLI.
 - 2 Add the OCPF BC plugin, following the install instructions in the repository README.
 - 3 Open your AL project folder.
 - 4 Type **/ocpf-bc:start**, or just ask to start a BC project with the OCPF framework.
-

/ocpf-bc:start

Or: *“Start a Business Central project with the OCPF framework.”*

github.com/ajansari/ocpfBcAgenticDevFramework

After one command

Helps you choose **Full or Lite**, and explains why

Installs the **latest runbook**, never a stale copy

Connects **Microsoft's AL tools**, with nothing to install

Adds **ready-made sub-agents** for the Light and Reasoning roles

Checks for **updates every session**, and asks before applying them

SIX SKILLS

start	status
al-standards	al-mcp-setup
notifications	update-framework



Open source. MIT licensed. Ready today.

WEB

ajansari.github.io/ocpfBcAgenticDevFramework

REPO

github.com/ajansari/ocpfBcAgenticDevFramework

VISIT

OnlyCopilotFans.com ·
OnlyBCFans.com ·
AJAnsari.com

EMAIL

aj@onlycopilotfans.com

AJ ANSARI · MICROSOFT MVP